
Parte 3: El Provider del Catálogo (Lectura de Datos)

1. El Concepto: Estado de Solo Lectura

En este punto, el catálogo de productos es una **fuentes de verdad** que rara vez cambia durante la sesión. Usaremos un `StreamProvider` o un `FutureProvider` generado para transformar la base de datos en un flujo de datos que Flutter pueda entender.

2. Implementación: Notifier del Catálogo (`providers/catalogo_provider.dart`)

Utilizaremos la anotación `@riverpod`. Este código buscará todos los productos que insertamos en la Fase 2.

```
import 'package:riverpod_annotation/riverpod_annotation.dart';
import 'package:isar/isar.dart';
import '../models/producto.dart';
import '../services/db_service.dart';

part 'catalogo_provider.g.dart';

@riverpod
class Catalogo extends _Catalogo {
  @override
  Future<List<Producto>> build() async {
    // 1. Accedemos a la instancia global de Isar que inicializamos en el main
    final isar = DBService.isar;

    // 2. Realizamos una consulta simple para traer todos los productos
    // Isar permite hacer esto de forma asíncrona y eficiente
    return await isar.productos.where().findAll();
  }

  // Opcional: Método para filtrar productos por categoría
  Future<void> filtrarPorCategoria(String categoria) async {
    state = const AsyncLoading();
    state = await AsyncValue.guard(() async {
      return await DBService.isar.productos
        .filter()
        .categoriaEqualTo(categoria)
        .findAll();
    });
  }
}
```

3. Teoría detrás del código

- **Future<List<Producto>> build():** Al ser una función asíncrona dentro de un **@riverpod**, Riverpod convierte automáticamente el estado en un **AsyncValue**. Esto significa que la UI sabrá automáticamente si los datos están cargando, si hubo un error o si ya están listos.
 - **isar.productos.where().findAll():** Esta es la sintaxis de Isar para un **SELECT * FROM productos**. Al no poner condiciones en el **where**, trae toda la colección.
 - **AsyncValue.guard:** Es una excelente práctica de ingeniería. Envuelve la operación en un bloque **try-catch** y, si algo falla, convierte el error en un estado de error que la UI puede manejar sin romperse.
-

4. Visualización en la UI (Previa)

Aunque aún no construimos la pantalla completa, así es como se usará este provider en un Widget:

```
// Ejemplo rápido de consumo
final catalogoAsync = ref.watch(catalogoProvider);

return catalogoAsync.when(
  data: (lista) => ListView(...), // Pintar productos
  loading: () => CircularProgressIndicator(),
  error: (err, stack) => Text('Error al cargar catálogo'),
);
```

5. Análisis:

Es importante entender por qué no usamos un simple **List<Producto>**.

"Al usar **AsyncValue**, eliminamos la necesidad de crear variables booleanas manuales como **isLoading = true**. Riverpod gestiona el ciclo de vida de la petición por nosotros, lo que hace que nuestro código sea declarativo en lugar de imperativo."
